Part 1: Deep Convolutional GAN (30 points)

ALL HYPERPARAMETERS

Model Hyperparameters:

- Image size = 64 (from 64 x 64)
- Convolution dimension = 32
- Noise size = 100 (from 100 x 1 x 1)
- Activation Functions:
    - Discriminator: ReLU applied after all convolution layers except the last one.
    - Generator: ReLU applied after all convolution layers except the last one, where Tanh was used.

Training Hyperparameters:

- Number of epochs = 500
- Bach size = 16
- Number of workers: 2
- Learning rate = 0.0002
- Beta 1 = 0.5 (Adam Optimizer Momentum Param)
- Beta 2 = 0.999 (Adam Optimizer Momentum Param)
- Optimizer: Adam
- Random Seed = 2 (tested values: 5, 6, 7, 8, 9, 10, 11)
- Loss Function:
    - Discriminator: Binary Cross-Entropy with Logits (torch.nn.BCEWithLogitsLoss())
    - Generator: Binary Cross-Entropy with Logits (torch.nn.BCEWithLogitsLoss())
- **Discriminator Training Label Trick: NOT USED!**
    - Real labels = 1 (used instead of 0.9 trick)
    - Fake labels = 0
- Fixed noise for sampling: (batch_size, noise_size, 1, 1), drawn from Uniform(-1, 1)
- Checkpoint Frequency = every 400 iterations
- Image Sampling Frequency = every 200 iterations
- Logging Frequency = every 10 iterations

Data Preprocessing and Augmentation Hyperparmeters:

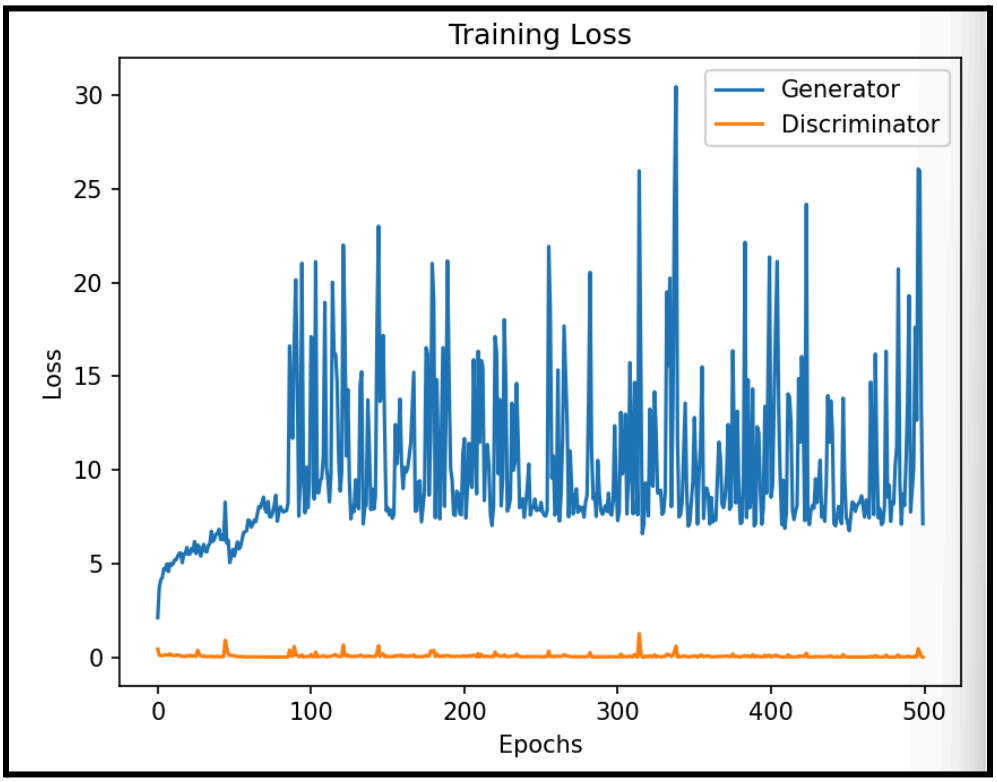
1) Basic Data Preprocessing

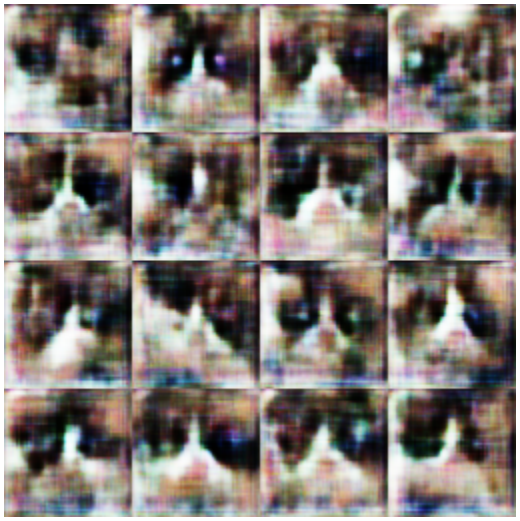
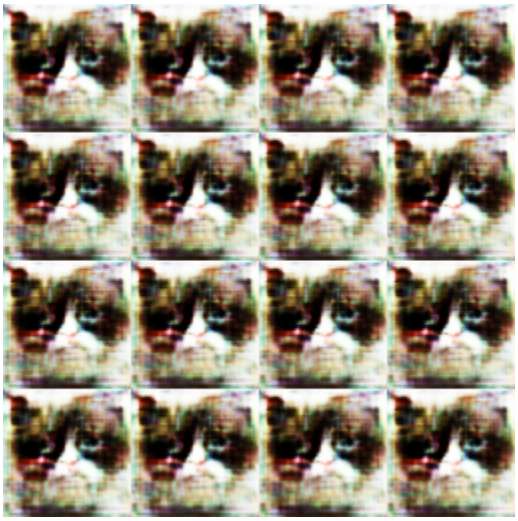
- Image resizing to 64 x 64 using Bicubic interpolation
- ToTensor() method for PyTorch compatibility
- Normalization: (scales images to range [-1, 1] for stable GAN training)
    - Mean: (0.5, 0.5, 0.5)
    - Std: (0.5, 0.5, 0.5)

2) Advanced Data Preprocessing
    - Resize images to 1.1× original size before cropping
    - Random Crop to 64x64
    - Random Horizontal Flip
    - Color Jitter:
        - Brightness: 0.03
        - Contrast: 0.05
        - Saturation: 0.07
        - Hue: 0.03
    - Commented out but used at some point
        - Gaussian Noise
            - (x + 0.05 * torch.randn_like(x)) → Disabled due to overfitting concerns
        - Normalization Consideration:
            - Attempted restricting normalization range to values in (0.4, 0.4, 0.4) / (0.3, 0.3, 0.3) to prevent image washout
        - Random Erasing
            - p=0.5, scale=(0.02, 0.2), ratio=(0.3, 3.3)

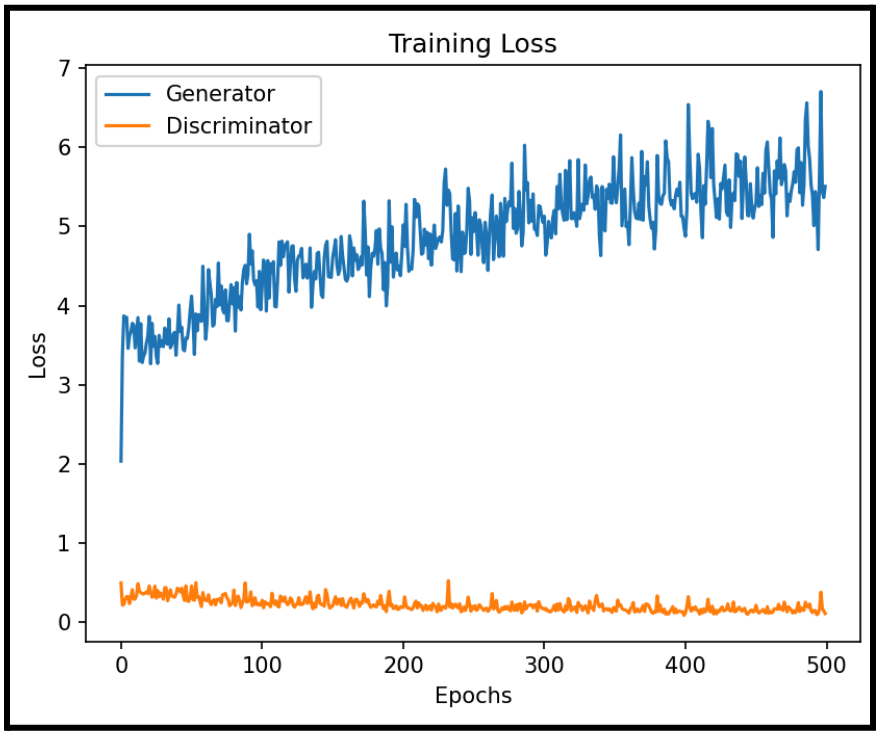
Training and Visualization

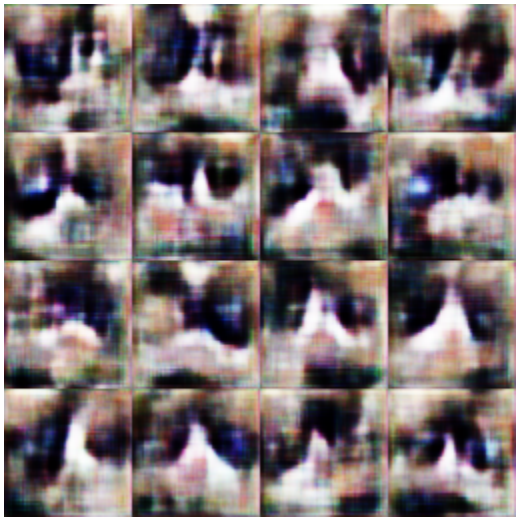
Basic Augmentation GAN Model



Iteration 200	Iteration 6400
	

Advanced Augmentation GAN Model



Iteration 200	Iteration 6400
	

Note: I was able to train a model that successfully avoided mode collapse. The generator was actually producing a variety of different cats that didn't come off as homogeneous; they looked similar to the real data. Surprisingly, this was achieved through removing the normalization step in the data augmentation, which caused the images to look washed out. Because I felt that the washed out images looked less realistic, I ended up not including them in the final report.

Part 2: Architectures and Objective Functions (20 points)

Implement Spectral Normalization Miyato et al. [2018]

Iteration 23400


Iteration 26000


I use the Adam optimizer, and I extended the number of epochs to give the model more chances to learn. I included both images, since I felt there were good characteristics regarding both. **Both images have much better resolution than the advanced augmentation.**

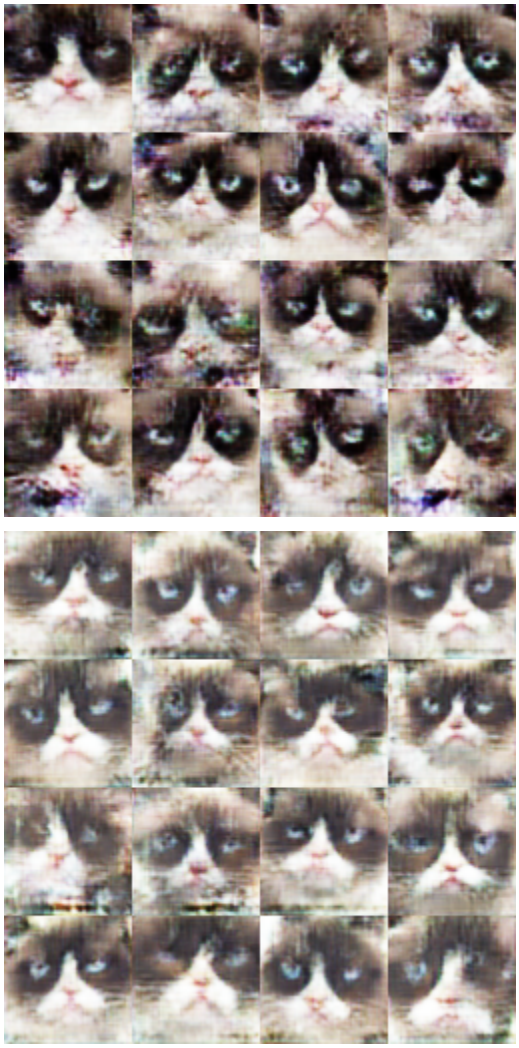
Implement Wasserstein GAN Arjovsky et al. [2017]

Iteration 8600


Of all the GANs, I definitely struggled the most with the WGAN. There was a level of noise that I struggled to remove, although it felt like this model was less susceptible to mode collapse compared to the others. Some things I tried were switching from the RMSprop optimizer to the Adam optimizer (after I realized that Adam was better for spectral normalization and RMSprop was better for weight clipping GANs). I tried increasing the number of epochs (to give the model a greater chance of converging) and decreasing beta_2 to improve the model's convergence ability. I also tried removing the color jitter from the advanced augmentations because I felt that removing it would also reduce the color noise introduced by the WGAN. I suspect that the color noise is also due to the spectral normalization, but I am not sure. I also tried changing n_critic, the number of times the generator/discriminator was updated, changing n_critic from 5 to 10. My logic behind this update was that the discriminator should be trained even more in order to provide a more accurate gradient flow between the discriminator and the generator. I also detached the fake images gradient from the computational graph during the discriminator update to avoid the generator from updating itself while the discriminator was updating (I forgot to add this in earlier). I also tried increasing the learning rate to see if the quality of the image would increase faster throughout each iteration. Overall, it was kind of challenging to test all these things with the WGAN because it took longer compared to the other GAN models to train.

If I had more time, I would have used weight clipping to enforce the Lipschitz constant rather than spectral normalization (which also enforces the Lipschitz constraint). I used spectral normalization because I thought it was superior in performance, and while it is, it might be more challenging to tune the WGAN hyperparameters such that it is effective. It may be that weight clipping is a more versatile strategy with any given set of hyperparameters, so it would have been a good use of my time to switch Lipschitz constrainment strategies.

Implement Least Squares GAN Mao et al. [2017]

Iteration 6400


1) Regular structure, just with implemented Spectral Normalization
2) Adjusted advanced augmentation: with Norm ([0.4,0.4,0.4], [0.3, 0.3, 0.3])

I started with the LS-GAN since it seemed to be the easiest to implement, and my starting strategy was to minimally modify the vanilla GAN architecture and running.py file. To start out, I only changed the loss functions to satisfy the least squares loss. However, the first cat picture was the result of that, so I knew that my model was struggling with noise. To compensate for this noise, I wanted steadier updates, so I tried switching from the Adam optimizer to RMSprop. RMSprop is more consistent in how it handles updating the gradients based on the loss. In hindsight, I think this was not the greatest idea. While it is true that the RMSprop is a steadier optimizer for a random time step, I think the Adam optimizer's main strength is being able to

adapt the strength of its adjustments based on the momentum term. While RMSprop may steadily update the model, this may not be enough to completely denoise the image unlike the Adam optimizer which would dynamically adjust the learning rate to compensate for not learning to denoise the image better. Another thing I tried doing was modifying one of the data augmentation steps. I wanted my LS GAN to suffer less from mode collapse (similar to the vanilla GAN), and I noticed that when I completely removed the normalization step from my vanilla GAN, I was actually able to see a variety of different cats, but the images were washed out, which meant that they were invalid as synthetic data. In an attempt to recreate this effect, I tried adjusting the normalization step to (mean=[0.4, 0.4, 0.4], std=[0.3, 0.3, 0.3]) because I felt it would be a good idea to tweak normalization instead of completely removing it. I wanted to see if I could replicate that success in the LS-GAN without resorting to removing the normalization step entirely. However, the images still appeared too washed out, suggesting that this approach wasn't as effective for the LS-GAN. Additionally, I experimented with detaching fake images during the discriminator update to prevent unwanted gradient updates in the generator.

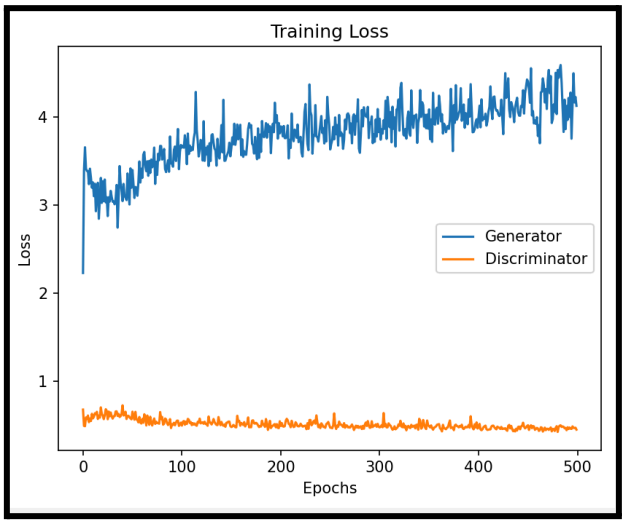
If I had more time, I would have definitely changed more hyperparameters. While I did change a few hyperparameters and leave my computer running overnight, I think I needed to do the same thing but with the Adam optimizer, as this would have likely led to higher resolution images. If I had more time, I would have also liked to experiment more with the idea of mimicking the effect of removing the normalization and beating the mode collapse issue without removing the normalization step. I am convinced that if I changed just a few hyperparameters or added more data augmentation steps, I could have potentially avoided the mode collapse while still maintaining strong contrast in the generated images.

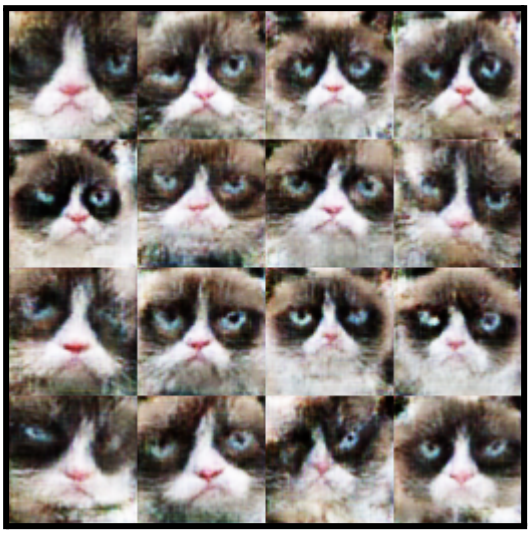
Two Additional GAN Training Techniques (With Papers Linked)

Technique #1: One-sided label smoothing

One-sided label smoothing, introduced by the paper *Improved Techniques for Training GANs*, is the idea that you don't have to feed the discriminator ground truth labels of 1 corresponding to the real images. At the beginning stage of training GANs, the discriminator often completely dominates the generator, so the idea of giving the discriminator a ground truth label of 0.9 instead of 1 is to give it more uncertainty, preventing it from becoming dominant immediately and giving more space for the ground truth model to catch up . Just by implementing this trick, the training stabilized significantly earlier, giving my GAN far more time to learn how to generate even higher resolution grumpy cat images.

Iteration 6400





This method performed better than the vanilla GAN, creating a higher resolution grumpy cat image.

Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., & Chen, X. (2016). *Improved techniques for training GANs*. arXiv preprint arXiv:1606.03498. https://doi.org/10.48550/arXiv.1606.03498

Technique #2: Top-k training

Top-k training, introduced by the paper *Top-k training of GANs: Improving GAN performance by throwing away bad samples*, is the idea that you don't have to propagate every gradient from the discriminator. You only have to propagate the gradients from the top k (you chose the number k) images that the discriminator/critic deems as realistic. The logic behind this is when updating the generator, you actually only have to focus on adjusting the model to finetune the generated images that performed the best, and the rest of the images will naturally follow. The alternative to this is training on gradients from images that are deemed extremely low quality, which would not benefit your training procedure since images that are basically "lost causes" will strongly influence the gradient, essentially diluting the more accurate and granular changes that would have only come from the top-k realistic images. Notably, while the generator only updates its weights from the top k realistic samples (as discerned from the discriminator), the discriminator is to learn from all the generated samples.

Iteration 7000


The Top-K model performed just about as well as the vanilla GAN with advanced augmentation did. I suspect this is because I needed to give the Top-K model more epochs to train for, as more granular/fine updates imply that the generator needs more updates in order to improve as much as it would with the vanilla. This is corroborated by the fact that it took longer for this GAN to remove noise, yet it showed consistent improvements in updates.

Sinha, S., Zhao, Z., Goyal, A., Raffel, C., & Odena, A. (2020). *Top-k training of GANs: Improving GAN performance by throwing away bad samples*. arXiv preprint arXiv:2002.06224. https://doi.org/10.48550/arXiv.2002.06224